Sentinel Asset Command

Military Asset Management System - Full-Stack Implementation Report

Project objective

Sentinel provides end-to-end asset visibility for vehicles, weapons, and ammunition across military bases. It calculates opening balances, net movements, assignments, expenditures, and closing balances while retaining an auditable record of every stock-changing operation.

Operational inventory equation

Measure	Calculation
Net movement	Purchases + Transfers in - Transfers out
Closing balance	Opening balance + Net movement - Assigned - Expended

System architecture

Layer	Implementation
Frontend	React with Vite, Tailwind CSS, Lucide icons, Recharts visualizations, Axios API client, protected routes, and role-sensitive navigation.
Backend	Node.js Express ES modules with Helmet, CORS, JSON input limits, JWT authentication, Bcrypt password verification, and modular controllers/routes.
Data	PostgreSQL through Prisma ORM. Relational foreign keys, unique stock balances, and base/equipment/date indexes protect integrity and reporting performance.
Deployment	Vercel frontend configuration plus a Render Blueprint for the Express API and managed PostgreSQL database.

Core flow

User signs in -> JWT carries user id, role, and base scope -> Express verifies the token -> RBAC restricts the route -> Prisma writes to PostgreSQL -> central audit service records the operation -> dashboard calculates balances from movement history.

Relational data design

The schema models Bases, Users, Equipment Types, Assets, Purchases, Transfers, Assignments, Expenditures, and Audit Logs. Asset is the current stock record and has a unique compound key of base and equipment type. Event tables remain immutable and support dynamic balance calculations.

Entity	Purpose and key controls
Base / User	Each Base Commander has an optional assigned base. Username is unique; role is constrained to ADMIN, BASE_COMMANDER, or LOGISTICS_OFFICER.
EquipmentType / Asset	Equipment categories are VEHICLE, WEAPON, or AMMUNITION. Asset uses a unique (baseId, equipmentTypeId) balance.
Purchase / Transfer	Inbound records and cross-base movements. Transfers retain source, destination, initiator, status, and timestamp.
Assignment / Expenditure	Record personnel/unit issue and asset consumption without overwriting history.
AuditLog	Records user, action, details, and timestamp for every purchase, transfer, assignment, and expenditure.

Transactional control

Transfers run in Prisma at Serializable isolation. The source Asset row is atomically decremented only when sufficient quantity exists. The destination Asset is upserted, the Transfer is created, and an AuditLog entry is written before the transaction commits. Any error rolls the entire transfer back.

Role-based access matrix

Capability	Administrator	Base Commander	Logistics Officer
Dashboard and inventory	All bases	Assigned base only	All bases
Log purchase	Yes	Assigned base only	Yes
Initiate transfer	Yes	No	Yes
Assign / expend	Yes	Assigned base only	No
View audit trail	Yes	No	No

API interface

Method	Endpoint	Purpose
POST	/api/auth/login	Verifies Bcrypt hash and returns a signed JWT with role and base scope.
GET	/api/dashboard	Calculates opening, purchases, transfers, assignments, expenditure, net movement, and closing balance.
GET	/api/assets	Returns current available Asset balances within the caller scope.
GET / POST	/api/purchases	Lists and creates inbound stock transactions.
GET / POST	/api/transfers	Lists and atomically completes cross-base stock transfers.
GET / POST	/api/assignments and /api/expenditures	Records unit allocation and consumed assets.
GET	/api/audit-logs	Administrator-only central mutation history.

Sample test accounts

Role	Username	Password	Scope
Administrator	admin_user	AdminPass123!	All bases
Base Commander	commander_alpha	CommandPass123!	Fort Alpha
Logistics Officer	logistics_officer	LogisticsPass123!	Purchases and transfers

Deployment and demonstration

For local development, Docker Compose starts PostgreSQL; Prisma db push creates the schema and the seed script creates the sample accounts and records. In production, Render provisions PostgreSQL and starts the API from render.yaml. Vercel builds the frontend and receives VITE_API_BASE_URL pointing to the Render /api endpoint. The Vercel preview retains a clearly labeled browser-demo fallback until that URL is configured.

Suggested 3-5 minute walkthrough

1. Log in as Administrator and explain dashboard filters and net movement modal. 2. Log a purchase and show inventory/audit updates. 3. Log in as Logistics Officer and execute a transfer while explaining the serializable transaction. 4. Log in as Base Commander and demonstrate forced Fort Alpha scope while creating an assignment. 5. Return as Administrator and show the central audit trail plus the Prisma/PostgreSQL schema.